

rag

A **measurable RAG system**, built as lab chapter 3 / curriculum §22 (*Build the First RAG System*): chunk -> embed -> index -> retrieve -> expand -> rerank -> contextualize -> build context -> generate a cited answer -> judge -> metrics, run over a generated **100-document, §7-metadata-carrying corpus** with **49 tier-balanced grounded questions**.

The lab instantiates the chapter’s thesis —

RAG = Information Retrieval + Context Engineering + Probabilistic Generation (§28)

— and its sharpest recurring claim: **semantic similarity is not relevance** (§4/§25). Two chunks can embed close to a query while only one *answers* it, so the system is built to expose *where* it failed rather than merely *that* it failed. The two-stage split of §2 ($D' = R(q, D)$, then $y \sim P(y \mid q, D')$) is the architecture: the retriever’s job is to **select evidence**, the model’s job is to **use it**, and the two are scored **separately** (§20 vs. §21).

Like chapters 1 and 2, the system splits on a **reliability boundary** — the shape the spec pins as I-009/R-20 (see *Status* for how tightly it is currently asserted):

- **Deterministic** (pure, offline, byte-reproducible, no LLM, no network, no embed model): the chunker, the in-memory vector store + cosine/BM25/hybrid/rerank math, the query expander, the contextual prefixing, the token-budget context builder, the citation/grounding gate, all metric math, and the corpus generator. Each probabilistic role has a deterministic double — **MockEmbedder** (FNV-1a hashed bag-of-words, O-1), **MockLLM**, **MockJudge**, **MockReranker**, **MockQueryExpander** — so **the entire suite runs offline**.
- **Probabilistic**: the **Embedder** (nomic-embed-text:latest) and the **LLM** (qwen3.8:27b-mlx), used in **generation + judging** by default. Expansion and reranking are *opt-in* LLM roles that default to their mocks (R-17/R-20).

The **central diagnostic** (R-15/I-008): every failed case is attributed to exactly one **failure_stage** — **chunking | retrieval | expansion | reranking | context | generation | judging** — so a report answers “*did retrieval provide the evidence?*” separately from “*did the model use it?*” (§21).

The authoritative description of behavior, contracts, invariants, edge cases, and acceptance criteria is **SPEC.md** (v0.2). See **SPEC.md §11 Traceability** for the **id -> where realized -> test** mapping. **SPEC_REVIEW_REPORT.md** is the 4-pass review whose findings F-001..F-016 were integrated into v0.2.

The §22 ladder: each capability is a flag, and the diff is the experiment

The CLI starts at a **dense baseline** and adds one capability at a time, re-running the *same* dataset so every architectural change becomes a measurable result (§22/§21).

Step	Flag	ch3		Watch
		§	What it targets	
baseline	<i>(all toggles off)</i>	§3/§4	dense cosine retrieval over chunks	recall@k, ndcg@k
+ metadata	carried in Chunk.meta	§7	recency/authority ranking + citation	which_field_decided
+ hybrid	--hybrid on [--alpha A]	§8	identifiers/error codes that embeddings blur	recall@k, mrr
+ reranking	--rerank on [--llm-rerank]	§9	fast top-N recall, then precise top-k	precision@k, ndcg@k
+ query expansion	--expand on [--n-expand N]	§10/§11	one phrasing misses the relevant chunk	recall@k on multi

Step	Flag	ch3 §	What it targets	Watch
+	<code>build-index</code>	§12	a chunk orphaned from its	<code>recall@k</code> on
contextual	<code>--contextual on</code>		document context	<code>chunking</code>
+ citations	<i>(always on)</i>	§13	unsupported claims / fabricated sources	<code>faithfulness</code> , <code>citation_quality</code>

Flag scoping (F-003) is the boundary that makes the experiment honest: `--strategy`, `--contextual`, `--chunk-size`, `--overlap`, `--embed-model` are **index-time** — a new value requires a `build-index rebuild` (a new index, not a new eval). `--hybrid`, `--rerank`, `--expand`, `--alpha`, `--k`, `--top-n`, `--model`, `--judge` are **query-time** — they recompute on the *existing* index.

Defaults (K-03, all CLI-overridable): `k=5`, `top_n=20`, `alpha=0.5`, every capability off, `strategy=heading`, `chunk_size=800` chars, `overlap=200`, `n_expand=3`, `seed=42`, `D_mock=256`.

Mechanics that are pinned, not hand-waved

```

INDEX TIME  Document(+meta) -> Chunker(strategy) -> Chunk -> Contextualizer -> Embedder
                                                    -> VectorStore (cosine)
                                                    -> BM25Index (lexical)

QUERY TIME  q -> dense top-N (+) BM25 top-N -> union, dedupe by chunk_id
            -> hybrid blend -> [expander: q -> {q1..qn} -> retrieve-per -> max-score union]
            -> reranker (top-N -> top-k) -> context builder (budget) -> Citer (gate)
            -> LLM.generate -> Answer{answer, confidence, citations[], status} (PROBABILISTIC)
            -> Judge.judge -> Verdict{correct, supported, faithfulness, ...} (PROBABILISTIC)
            -> metrics.retrieval(G, R_k) | metrics.generation(verdict) -> report

```

- **Hybrid (R-04/O-3)**: `score = alpha * norm(s_sem) + (1 - alpha) * norm(s_lex)` over **per-channel min-max-normalized** scores within the query, so the channels are commensurable; `alpha=1` is pure dense, `alpha=0` pure lexical; a candidate absent from a channel carries raw 0.0; a *zero-range* channel normalizes everyone to 1.0 and ties break by `chunk_id` ascending — never a 0/0.
- **Rerank (R-05)**: `Reranker.rerank(q, candidates) -> ScoredChunk[]` with `k <= N`; `MockReranker` is `0.6 * coverage + 0.4 * norm-cosine`, `LLMReranker` is the opt-in substitute behind the same seam. Final order: `.rerank desc, chunk_id asc` (F-012).
- **Context budget (I-004/I-005)**: `est_tokens(s) = ceil(len(s)/4)` is the *single* estimator shared by builder and report, folded as a **running sum over included docs** — so reported `context_tokens` is what was built, and `truncated=True` iff a doc was dropped.
- **Grounding gate (R-08/I-003)**: any cited `chunk_id` outside `Context.provenance` is **deterministically dropped and flagged** (`grounding_violation=True`, `supported=False`) — the model is never trusted to self-ground. Retrieved text is **data, not instructions** (R-21): an injection-pattern hit sets `injection_warning` and raises the INJECTION! banner, distinct from the runtime banners.
- **Schema gate (R-09/R-10/I-010)**: raw text -> strip an optional Markdown JSON fence -> `json.loads -> jsonschema validate -> accept or reject-and-retry`; an out-of-range `confidence` or a missing required field can never reach `COMPLETED`.

Two metric families, measured separately (§20 vs. §21)

Family	Metrics	Reference
retrieval	<code>precision@k</code> , <code>recall@k</code> , <code>mrr</code> , <code>map</code> , <code>ndcg@k</code>	each question's <code>relevant_chunks = G</code>

Family	Metrics	Reference
generation	faithfulness = supported / total_claims, completeness = reflected_gold_facts / total, citation_quality = relevant / total citations	the Verdict, <i>after</i> the Citer's recount (F-005)

The math is asserted against **worked examples** (I-001): $G=\{c1,c3,c5\}$, $R_5=[c1,c8,c3,c9,c5] \rightarrow P=0.60$, $R=1.0$, $MRR@5=1.0$, $NDCG@5=0.88547$. Every denominator has a declared empty behavior (I-007): $\text{precision}=\text{None}$ when $TP+FP=0$, $\text{recall}=\text{None}$ when $TP+FN=0$, $\text{mrr}@k=0.0$ when nothing relevant is in the top-k, $\text{ndcg}=\text{None}$ when $IDCG=0$, and the generation ratios $=0.0$ at zero claims — a no-retrieval row contributes nothing to a mean. Aggregates carry **by_tier** and **by_capability** (I-012), plus **failure_breakdown**.

Seven failure-mode tiers (R-13) — every chapter failure is a *measured* row

Tier	Probes	ch3 §
easy	one chunk answers it — the sanity floor	§14
multi	A <i>and</i> B <i>and</i> C: set retrieval, not multi-hop agentic search	§19
chunking	a rule split from its governing condition by a boundary	§14
distractor	lexically similar, semantically irrelevant	§15
conflict	disagreeing policies; resolve by version > updated_at > access_level	§16
recency	dated versions; newest wins	§17
injection	an adversarial payload riding inside retrieved evidence	§18

Availability: three outcomes, never a crash (R-19/F-013)

Outcome	Condition	Banner	Exit
RUN_REAL	daemon up, both models pulled	none	0
DEGRADED MOCK	--mock forced or daemon unreachable	[REAL>MOCK] ... -> doubles keep running	0
PULL_REQUIRED	daemon up, model not pulled	MODEL_MISSING: run ollama pull <m>	4

(One wording bug: the forced--mock case reuses the *unreachable* banner, so a deliberate --mock on on a live daemon still prints **Ollama unreachable** — see Status.)

Other CLI exit codes (§5.1): 2 bad usage (e.g. --top-n < --k, E-15), 3 corpus / questions / index **load** failure (E-01/E-14/I-013 — a **relevant_chunks** id absent from the built index is a *load-time* error, never a silent 0-recall). Per-case faults are **recorded rows**, not failing exits.

Requirements

- Python **3.12** (managed by uv). **uv sync --all-extras** is the setup command: the test deps (pytest, pytest-qt) and PyQt5 are *optional extras* in pyproject.toml, so a plain **uv sync** yields a CLI-only env and would even *remove* an existing pytest.
- **No Ollama, no network, no embed model** for the default --mock path or the whole test suite. The real backends — served by Ollama at <http://localhost:11434> — are an **opt-in manual smoke** (§9.11/K-05).
- PyQt5 is an **optional** extra; the suite never needs Qt (and runs **offscreen** when it is).

The four CLI subcommands (`rag --help`)

Command	Scope	What it does	Output
<code>gen-corpus</code>	<code>offline</code>	deterministic corpus + question generator (R-13/T-01); <code>--seed</code> is the only thing it governs	<code>documents/corpus.jsonl</code> + <code>questions.json</code>
<code>build-index</code>	<code>index-time</code>	chunk (<code>--strategy/--chunk-size/--overlap</code>) -> contextualize (<code>--contextual</code>) -> embed (<code>--embed-model</code> , or the mock) -> insert into VectorStore + BM25Index	a one-line summary: built index: 100 chunks strategy=heading contextual=False overlap=200 mock=True
<code>eval</code>	<code>query-time</code>	the full §22 pipeline over <code>questions.json</code> -> per-case <code>RunMetrics</code> -> <code>AggregateMetrics</code>	human summary on stdout + <code>report.json</code>
<code>show</code>	<code>query-time</code>	one case's retrieved ranking + <code>status</code> + <code>failure_stage</code> ("what world did the model see?")	that one row, printed

build-index writes nothing to disk. It builds the (VectorStore, BM25Index) in memory, prints the chunk count and the *resolved* index-time flags, and exits – there is no pickle cache and no `--index` path flag. So it is a **probe** (“how many chunks did these index-time flags produce?”), **not** a prerequisite for `eval`: `eval` and `show` each rebuild the index from `--corpus` on whatever index-time flags *they* are handed (F-003 is satisfied because `eval` can never see a stale index – it always rebuilds). A full mock rebuild is ~0.06s, so the SPEC §5.1 pickle cache is an unimplemented option under no budget pressure (Status).

To make the §14 “+contextual” diff, put `--contextual on` on the `eval` that produces the report – not (only) on a separate `build-index` whose output is thrown away.

Development setup

optional, for the REAL path only (the mock path needs neither):

```
# ollama pull nomic-embed-text      # embedding
# ollama pull qwen3.8:27b-mlx      # generation + judge (ID 5642e97495e1, ~18GB)
```

```
uv sync --all-extras                # .venv (3.12) + dev (pytest, pytest-qt) + gui (PyQt5)
uv run rag gen-corpus --seed 42      # -> documents/corpus.jsonl + questions.json (<1s, T-01)
uv run pytest                        # the SPEC §9 suite -- FULLY OFFLINE (K-01/I-011)
```

```
uv run rag eval --mock on            # §22 baseline -> report.json (<5s, K-02)
uv run rag eval --mock on --hybrid on --alpha 0.3 # +hybrid on the SAME index -> by_capability diff
uv run rag eval --mock on --rerank on --expand on # query-time toggles: no rebuild needed
uv run rag show --qid q-chunking-00    # one case's ranking + status + failure_stage
```

*# index-time: change what the *eval* rebuilds -- pass the flag to the eval that reports (F-003)*

```
uv run rag build-index --mock on --contextual on # PROBE: chunk count + resolved flags; writes nothing
uv run rag eval --mock on --contextual on        # the report: eval rebuilds the contextual index
```

```
uv run rag eval --mock off --model qwen3.8:27b-mlx --embed-model nomic-embed-text:latest # REAL (opt-i)
uv run rag-gui                                # optional GUI surface (see Status)
```

Toggle syntax: `--mock`, `--hybrid`, `--rerank`, `--expand`, `--contextual`, `--judge` and `--show-banners` are `type=on|off` **value** options, not bare switches — write `--mock on`, never `--mock` alone (argparse rejects it with exit 2). Accepted values are `on|true|1|yes` and `off|false|0|no`, and `--mock` already defaults

to on, so an offline eval can omit it entirely. --llm-rerank and --llm-expand are bare flags but currently unwired (Status).

A --mock eval prints the human summary and writes the machine-readable report:

```
RAG eval report -- per-metric means over non-None rows
precision@5: 0.0082
recall@5: 0.0306
citation_quality: 1.0000
by_tier:
  chunking (n=7): {'precision': 0.0, 'ndcg': 0.0}
  easy (n=7): {'precision': 0.0285, 'ndcg': 0.0615}
schema_validation: jsonschema
```

Those absolute values are a property of the hashed bag-of-words MockEmbedder over a synthetic corpus, **not** a quality claim; the artifact is the *per-capability diff* on the same dataset and the *per-tier* attribution of where a case failed.

Layout (as built, derived from SPEC.md §3–§5)

```
src/rag/
types.py          # C-01/C-09/C-10: Document, Chunk, ScoredChunk, Citation, Question,
                  # Answer, Verdict, RunMetrics, AggregateMetrics, CaseState
corpus.py         # C-01 load_corpus / load_questions + generate_corpus_and_questions [no LLM]
chunking.py       # C-03 Fixed|Heading|ContextualChunker (embed_text prefix) +
                  # boundary_guard (split_risk, I-013) [no LLM]
embedding.py      # C-02 MockEmbedder (O-1 FNV-1a hashed-BoW) + OllamaEmbedder [PROBABILISTIC]
retrieval.py      # C-02 VectorStore/cosine + BM25Index, C-04 HybridRetriever,
                  # C-05 MockReranker [no LLM]
expand.py         # C-06 MockQueryExpander + LLMQueryExpander + multi_query union [no LLM by default]
context.py        # C-03 build_context: dedupe + token budget + provenance, est_tokens [no LLM]
citation.py       # C-08 Citer: grounding gate + claim extraction + injection scan [no LLM]
metrics.py        # C-11 P/R@k, MRR, MAP, NDCG + faithfulness/completeness/citation_quality
                  # + aggregate (by_tier / by_capability) [no LLM]
schemas.py        # R-09/R-10/I-010 answer + verdict gate (jsonschema, structural fallback)
model.py          # C-09 MockLLM + OllamaLLM + LLMReranker [PROBABILISTIC]
judgment.py       # C-10 MockJudge + OllamaJudge [PROBABILISTIC]
pipeline.py       # C-12 build_index (chunk -> contextualize -> embed -> insert) /
                  # run_case / run_dataset (§3.1–§3.3 FSM, failure_stage R-15)
availability.py   # R-19/E-13 DEGRADED MOCK | PULL_REQUIRED | RUN_REAL -> banner + exit code
app.py            # §5.1 the `rag` CLI (build-index / eval / gen-corpus / show) + entry points
ui.py             # §5.2 the `rag-gui` surface (see Status)
logging_setup.py  # loguru --verbose / --quiet wiring
schemas/          # answer.json, verdict.json -- the two structured-output contracts
tests/            # SPEC §9: 18 files / 150 tests, fully offline
tools/reindent.py # house-style indent normalizer (4/8/12 grid) with an ast.parse safety guard
documents/, questions.json, report.json # generated by gen-corpus / eval; not committed
```

httpx is imported **lazily**, **inside** the three real-backend call sites (embedding.py, model.py, availability.py), and judgment.py reaches the daemon through model.OllamaClient — so importing the package or running the suite needs no network stack in the deterministic layers (R-17/I-011).

Tests

uv run pytest is the §9 acceptance suite: **18 files / 150 tests, fully offline, ~0.2s** against the K-01 budget of < 90s. They map T-01..T-23 onto the contracts — corpus and ground-truth loading, dense/hybrid/BM25

ranking, expansion, chunk-boundary recovery, the metric worked examples and no-/0 guards, the token budget, the schema gate, the grounding gate and injection scan, the five doubles, the per-case FSM and failure attribution, the I-009 source scan, CLI exit codes, the availability taxonomy, the `--verbose/--quiet` loguru wiring, the embed request/response contract, the judge verdict bounds/retry path, and the GUI degrade path. The real Ollama path is **only** the opt-in §9.11 smoke and never runs in `uv run pytest` (I-011/K-05). It was live-smoke-verified (2026-08-30) after the `/api/embed` request-field fix: `nomic-embed-text:latest` returned L2-normalized 768-d embeddings (~0.4s/call, 100-chunk index in seconds), and a single real `show` (dense retrieval + `qwen3.8:27b-mlx` generation) ran to **SCORED**, exit 0, in 9s. (Retrieval’s `c > 0.0` cosine filter means the real path may legitimately return fewer than `k` candidates — only positively-similar chunks are ranked.)

Status (implementation vs. SPEC v0.2)

Everything above is implemented and green **except** these known divergences:

- **build-index is a no-op probe (SPEC §5.1 “May persist the index to a pickle” is unimplemented).** `cmd_build_index` builds the index, prints the chunk count, and discards it — no pickle cache, no `--index` path; a following `eval` rebuilds from `--corpus`. The design stays safe only because every command rebuilds, and no CLI test covers the command itself (T-03 exercises `pipeline.build_index` directly, not the `build-index` subcommand).
- **SPEC.md §10’s repro block is wrong as written.** It uses `uv sync` (which does not install the `dev/gui` extras, so `uv run pytest` fails on a fresh clone) and bare `rag eval --mock` (which `argparse` rejects, exit 2). Corrected above; the spec text still needs the same fix.
- **The DEGRADED MOCK banner lies when --mock is forced.** `resolve_availability` short-circuits on `mock` and returns the *unreachable* banner, so `--mock` on prints **Ollama unreachable** even on a box where Ollama is running. R-19/F-013 asks for distinct banners per condition; a forced-mock case is not the same fact as an unreachable daemon.
- **rag-gui is a headless console fallback, not the §5.2 workbench.** `ui.py` probes for a Qt binding and then delegates to the CLI `show` view (R-16/F-012: it must degrade, never block on stdin, never crash). The widget panel — ranked->reranked->contextualized evidence with per-stage scores, the truncation badge, the verdict pills, the INJECTION! badge — is **not built**; PyQt5 stays an optional extra and `tests/test_ui.py` asserts only the degrade-to-console contract.
- **Three CLI flags are parsed but not wired:** `--llm-rerank` and `--llm-expand` (so `rerank/expand` always use their deterministic doubles) and `--stop-on-error`.
- **The I-009 guard is softer than the spec.** `tests/test_source_scan.py` (T-02, as re-pointed) scans §4 *interface existence and naming*, not “no Ollama/httpx/model name in these files” — and `expand.py` carries a default `qwen3.8:27b-mlx` string for the opt-in `LLMQueryExpander`. The boundary holds by construction today; it is not asserted.
- **Corpus artifacts differ from T-01’s letter.** `gen-corpus` writes one `documents/corpus.jsonl` (100 docs, §7 metadata inline) rather than 100 `documents/NNN.txt` files, and the default question count lands on **49** (7 per tier × 7 tiers) rather than the 25 named in §9.1. `load_corpus` accepts a directory *or* a `.jsonl`, so `--corpus` works either way.
- **by_capability is in the JSON report but not the stdout summary.** §5.1.1 asks the human summary to carry the per-capability diff; `_summary_lines` prints per-metric means, `failure_breakdown` and `by_tier`, so the diff currently means reading `report.json`.
- **Determinism nuance (F-016).** `--mock` defaults to **on**, and the doubles are input-determined (they consume no seed/RNG); `--seed` governs `gen-corpus` only. Two identical mock runs are byte-identical without any RNG.